

Project 3

Centralized Secret Management System using AWS Secrets Manager and IAM Policies

Scenario

You joined as a DevSecOps Engineer. During an internal security audit, it was discovered that developers are storing database passwords inside application configuration files.

The CISO instructs:

"Eliminate hardcoded secrets and implement centralized secure secret management with controlled access."

Objective

Implement a secure secrets management system where:

Application retrieves secrets dynamically

No credentials stored in code

Access controlled via IAM roles

Project Requirements

1. Application Setup

Deploy EC2 instance

Install simple application that connects to database

Initially demonstrate hardcoded secret (for comparison)

2. Secrets Manager Setup

Store:

DB username

DB password

Enable automatic rotation (optional)

3. IAM Role Configuration

Create IAM role for EC2

Grant least privilege access to specific secret

Attach role to instance

4. Application Modification

Modify application to:

Fetch secrets from Secrets Manager using SDK

Remove hardcoded credentials

5. Security Validation

Verify:

Application works without stored credentials

No access key configured on server

Technologies & Tools

EC2

AWS Secrets Manager

IAM

Python / Node.js

Deliverables

Application code

IAM policy JSON

Screenshot of secret retrieval

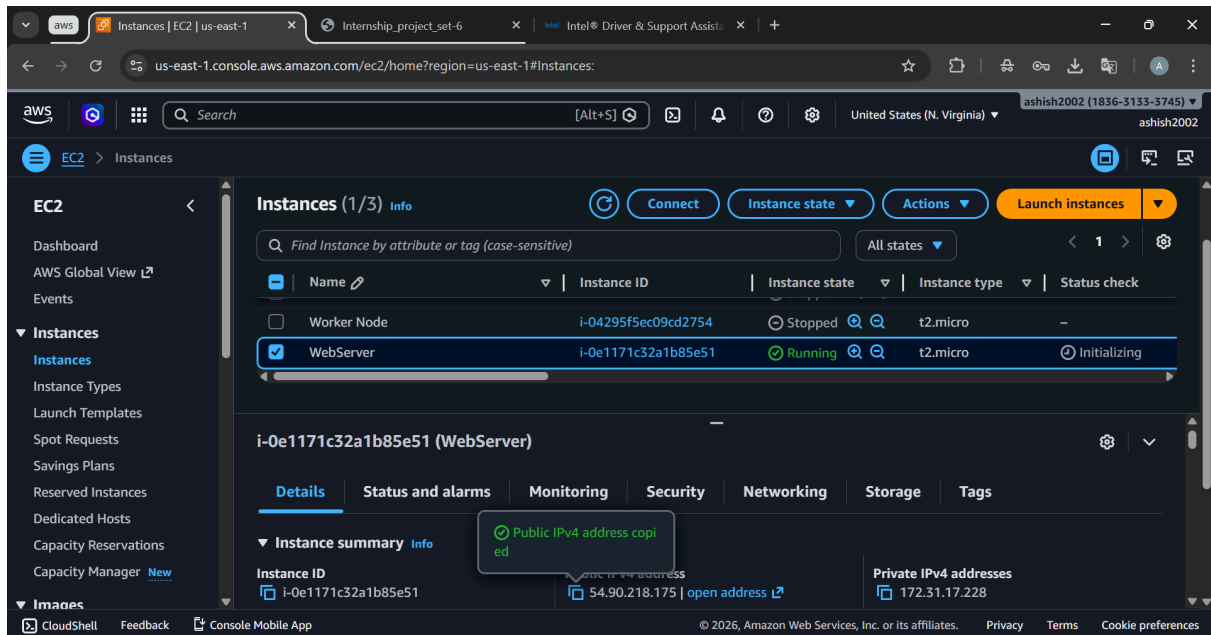
README explaining

Why secret rotation matters

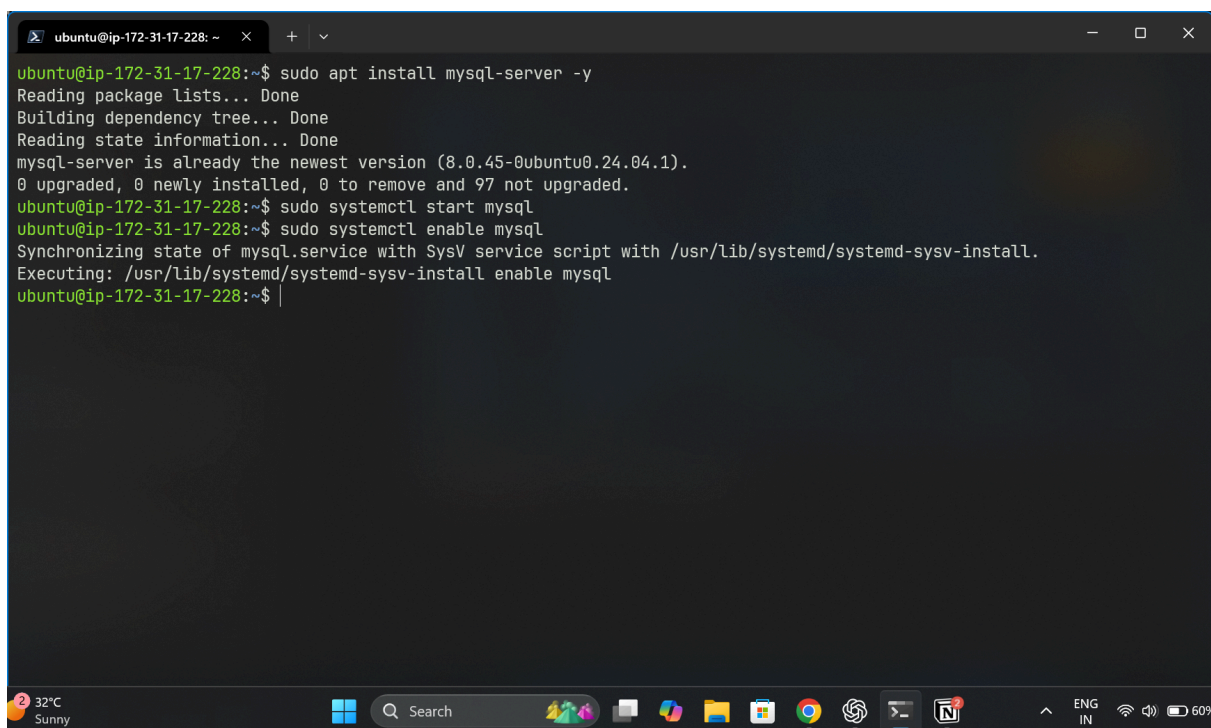
Security improvements

Solution :

Launch the EC2 Server.



Install Mysql-server, start and enable it.



Install Nodejs and NPM.

```
ubuntu@ip-172-31-17-228:~$ sudo apt install nodejs -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libcares2 libnode109 node-acorn node-busboy node-cjs-module-lexer node-undici node-xtend nodejs-doc
Suggested packages:
  npm
The following NEW packages will be installed:
  libcares2 libnode109 node-acorn node-busboy node-cjs-module-lexer node-undici node-xtend nodejs nodejs-doc
0 upgraded, 9 newly installed, 0 to remove and 97 not upgraded.
Need to get 16.1 MB of archives.
After this operation, 70.4 MB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libcares2 amd64 1.27.0-1.0ubuntu1 [73.7 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 node-xtend all 4.0.2-3 [3902 B]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 nodejs amd64 18.19.1+dfsg-6ubuntu5 [306 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 node-acorn all 8.8.1+ds+~cs25.17.7-2 [115 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 node-cjs-module-lexer all 1.2.3+dfsg-1 [32.1 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 node-busboy all 1.6.0+~cs2.6.0-2 [17.3 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 node-undici all 5.26.3+dfsg1+~cs23.10.12-2 [325 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 libnode109 amd64 18.19.1+dfsg-6ubuntu5 [11.6 MB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 nodejs-doc all 18.19.1+dfsg-6ubuntu5 [3552 kB]
Fetched 16.1 MB in 0s (71.4 MB/s)
Selecting previously unselected package libcares2:amd64.
(Reading database ... 87952 files and directories currently installed.)
Preparing to unpack .../0-libcares2_1.27.0-1.0ubuntu1_amd64.deb ...
Unpacking libcares2:amd64 (1.27.0-1.0ubuntu1) ...
Selecting previously unselected package node-xtend.
Preparing to unpack .../1-node-xtend_4.0.2-3_all.deb ...
```

```
ubuntu@ip-172-31-17-228: ~/$  
ubuntu@ip-172-31-17-228: ~/backend$ sudo apt install npm  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
binutils binutils-common binutils-x86-64-linux-gnu build-essential bzip2 cpp cpp-13 cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu  
dpkg-dev eslint fakeroot g++ g++-13 g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu  
gcc-14-base gcc-x86-64-linux-gnu gyp handlebars javascript-common libalgorithm-diff-perl libalgorithm-diff-xs-perl  
libalgorithm-merge-perl libao3 libasan8 libatomic1 libauthen-sasl-perl libbinutils libc-bin libc-dev-bin libc-devtools libc6  
libc6-dev libc6i-0 libcrypt-dev libctf-nbfd0 libctf0 libdata-dump-perl libdeb265-0 libdpkg-perl libegl-mesa0 libegl1 libfakeroot  
libfile-basedir-perl libfile-desktopentry-perl libfile-fcntllock-perl libfile-listing-perl libfile-mimeinfo-perl libfont-afm-perl  
libgcc-13-dev libgcc-s1 libgd3 libgles2 libgomp1 libgprofng0 libheif-plugin-aomdec libheif-plugin-aomenc libheif-plugin-libde265  
libheif1 libhtml-form-perl libhtml-format-perl libhtml-tree-perl libhttp-cookies-perl libhttp-daemon-perl libhttp-negotiate-perl  
libhwasan0 libio-socket-ssl-perl libio-stringy-perl libipc-system-simple-perl libisl23 libitm1 libjs-async libjs-events  
libjs-inherits libjs-is-typedarray libjs-jquery libjs-regenerate libjs-source-map libjs-sprintf-js libjs-typedarray-to-buffer  
libjs-util liblsan0 liblwp-protocol-https-perl libmailtools-perl libmpc3 libnet-dbus-perl libnet-http-perl libnet-smtp-ssl-perl  
libnet-ssleay-perl libnode-dev libnotifify-bin libnotifify4 libquadmath0 libre2-10 librsync1 libssl-dev libssl3t64 libstdc++-13-dev  
libstdc++6 libtie-ixhash-perl libtiny-perl libtins2 libubsan1 libuv1-dev libwww-perl libwww-robotrules-perl  
libx11-protocol-perl libxml-parser-perl libxml-twig-perl libxml-xpather-engine-perl linux-libc-dev linux-tools-common locales  
lto-disabled-list make manpages-dev node-agent-base node-agent-base node-ajv node-ajv-keywords node-ampproject-remapping  
node-ansi-escapes node-ansi-regex node-ansi-styles node-anymatch node-aproba node-archy node-are-we-there-yet node-argparse  
node-arrify node-assert node-async node-async-each node-auto-bind node-babel-helper-define-polyfill-provider  
node-babel-plugin-add-module-exports node-babel-plugin-lodash node-babel-plugin-polyfill-corejs2  
node-babel-plugin-polyfill-corejs3 node-babel-plugin-polyfill-regenerator node-babel7 node-babel7-runtime node-balanced-match  
node-base node-base64-js node-binary-extensions node-brace-expansion node-braces node-browserslist node-builtins node-cache  
node-cache-base node-camelcase node-canuse-lite node-chalk node-chokidar node-chownr node-chrome-trace-event node-ci-info  
node-cli-boxes node-cli-cursor node-cli-table node-cli-truncate node-clui node-clone node-clone-deep node-collection-visit  
node-color-convert node-color-name node-colors node-columnify node-commander node-commander node-concat-stream  
node-console-control-strings node-convert-source-map node-copy-concurrently node-core-js node-core-js-compat node-core-js-pure
```

Run the Bash Script which install the express, MySQL dependencies and also create the structure of the project.

```
#!/bin/bash

PROJECT="vite-node-mysql-app"

echo "Creating project $PROJECT ..."
```

```

mkdir $PROJECT
cd $PROJECT

#####
# BACKEND
#####

mkdir backend
cd backend

npm init -y > /dev/null
npm install express mysql2 cors > /dev/null

cat <<EOF > server.js
const express = require("express");
const mysql = require("mysql2");
const cors = require("cors");

const app = express();
app.use(cors());

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "ashish@2002",
  database: "testdb"
});

app.get("/api/status", (req, res) => {

  db.connect((err) => {

    if (err) {
      res.json({
        status: "FAILED",
        message: "Backend running but MySQL connection failed"

```

```

    });
  } else {
    res.json({
      status: "SUCCESS",
      message: "Backend + MySQL Connected Successfully"
    });
  }

});

});

const PORT = 5000;

app.listen(PORT, () => {
  console.log("Server running on port " + PORT);
});
EOF

cd ..

#####
# FRONTEND
#####

npm create vite@5 frontend -- --template react > /dev/null

cd frontend
npm install > /dev/null
npm install axios > /dev/null

cat <<EOF > src/App.jsx
import { useEffect, useState } from "react";
import axios from "axios";

function App() {

  const [message, setMessage] = useState("Checking backend

```

```

connection...");

useEffect(() => {
  axios.get("http://localhost:5000/api/status")
    .then(res => {
      setMessage(res.data.message);
    })
    .catch(() => {
      setMessage("Cannot connect to backend");
    });
}, []);

return (
  <>
    <h1>Connection Status</h1>
    <h2>{message}</h2>
  </>
);
}

export default App;
EOF

cd ..

echo ""
echo "Project created successfully!"
echo ""
echo "Run backend:"
echo "cd $PROJECT/backend"
echo "node server.js"
echo ""
echo "Run frontend:"
echo "cd $PROJECT/frontend"
echo "npm run dev"

```

```
ubuntu@ip-172-31-17-228: ~/ | x + v
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend$ ls
README.md  dist  eslint.config.js  index.html  node_modules  package-lock.json  package.json  public  src  vite.config.js
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend$ npm run build

> frontend@0.0.0 build
> vite build

vite v5.4.21 building for production...
✓ 81 modules transformed.
dist/index.html      0.46 kB | gzip: 0.30 kB
dist/assets/index-kQJbKSsj.css  0.92 kB | gzip: 0.50 kB
dist/assets/index-Ci0AeB_9.js 180.07 kB | gzip: 60.51 kB
✓ built in 2.50s
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend$ sudo cp -r dist/* /var/www/html/
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend$ |
```

Now set MySQL password and create the database called testdb.

```
mysql> ALTER USER 'root'@'localhost'
-> IDENTIFIED WITH mysql_native_password
-> BY 'ashish@2002';
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> create database testdb
-> ;
Query OK, 1 row affected (0.01 sec)
```

Run the Frontend

```
npm run build
```

Now Install the Nginx and copy all the files from /frontend/dist/* to the nginx default location /var/www/html.

```
ubuntu@ip-172-31-17-228: ~/ + v
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend$ npm run build

> frontend@0.0.0 build
> vite build

vite v5.4.21 building for production...
✓ 34 modules transformed.
dist/index.html      0.46 kB | gzip: 0.30 kB
dist/assets/react-CHdo91hT.svg 4.13 kB | gzip: 2.14 kB
dist/assets/index-n_ryQ3BS.css 1.39 kB | gzip: 0.72 kB
dist/assets/index-ekARzf_2.js 143.20 kB | gzip: 46.03 kB
✓ built in 1.96s
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend$ ls
README.md dist eslint.config.js index.html node_modules package-lock.json package.json public src vite.config.js
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend$ cd dist/
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend/dist$ ls
assets index.html vite.svg
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend/dist$ sudo apt install nginx -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  nginx-common
Suggested packages:
  fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  nginx nginx-common
0 upgraded, 2 newly installed, 0 to remove and 88 not upgraded.
Need to get 565 kB of archives.
```

```
ubuntu@ip-172-31-17-228: ~/ + v
Running kernel seems to be up-to-date.

Restarting services...

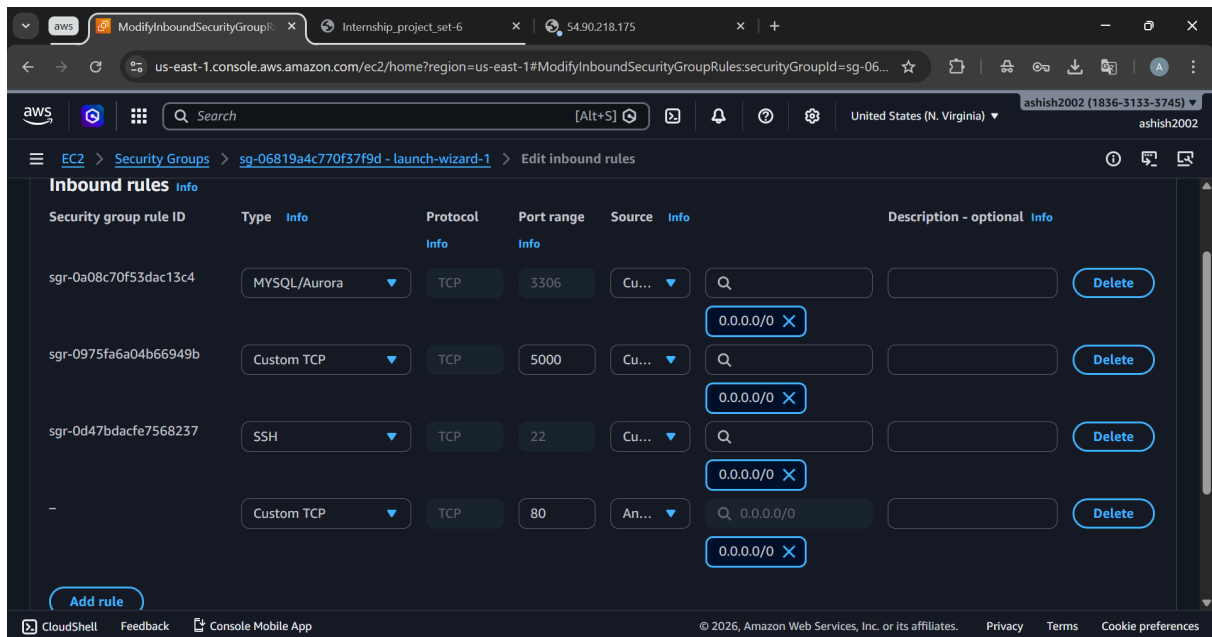
Service restarts being deferred:
/etc/needrestart/restart.d/dbus.service
systemctl restart getty@tty1.service
systemctl restart networkd-dispatcher.service
systemctl restart serial-getty@ttyS0.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service

No containers need to be restarted.

User sessions running outdated binaries:
ubuntu @ session #1: sshd[932,1046]
ubuntu @ user manager service: systemd[937]

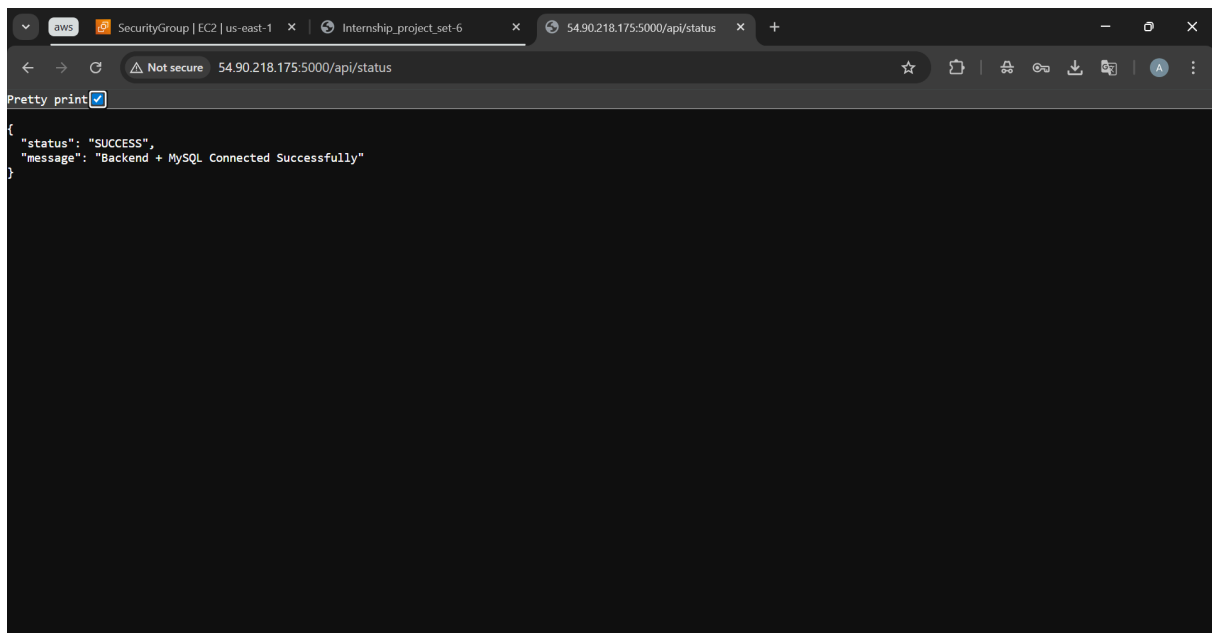
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend/dist$ ls
assets index.html vite.svg
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend/dist$ sudo systemctl start nginx
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend/dist$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend/dist$ sudo cp * /var/www/html/
cp: -r not specified; omitting directory 'assets'
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend/dist$ sudo cp -r * /var/www/html/
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend/dist$ |
```

In inbound security group enable the port 80 , 3306, 5000.



Run Backend

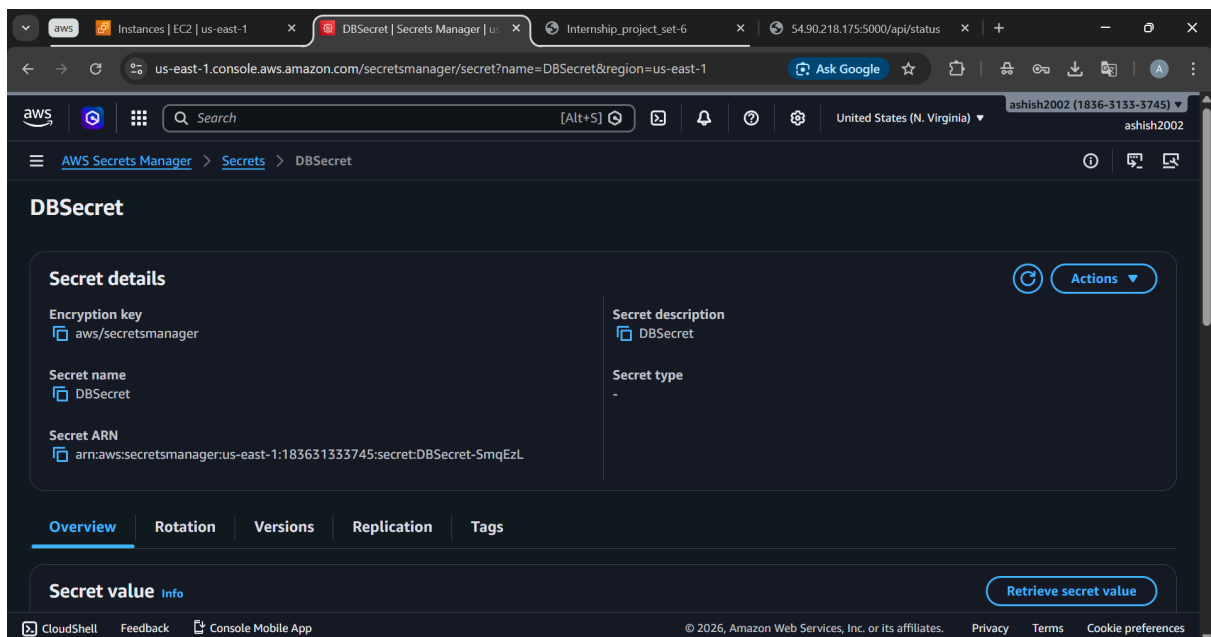
```
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend$ ls
README.md dist eslint.config.js index.html node_modules package-lock.json package.json public src vite.config.js
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend$ cd dist/
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend/dist$ ls
assets index.html vite.svg
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend/dist$ sudo cp -r * /var/www/html/
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/frontend/dist$ cd ../../
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app$ ls
backend frontend
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app$ cd backend/
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/backend$ ls
controllers db node_modules package-lock.json package.json routes server.js
ubuntu@ip-172-31-17-228:~/vite-node-mysql-app/backend$ node server.js
Server running on port 5000
Connected to MySQL
```



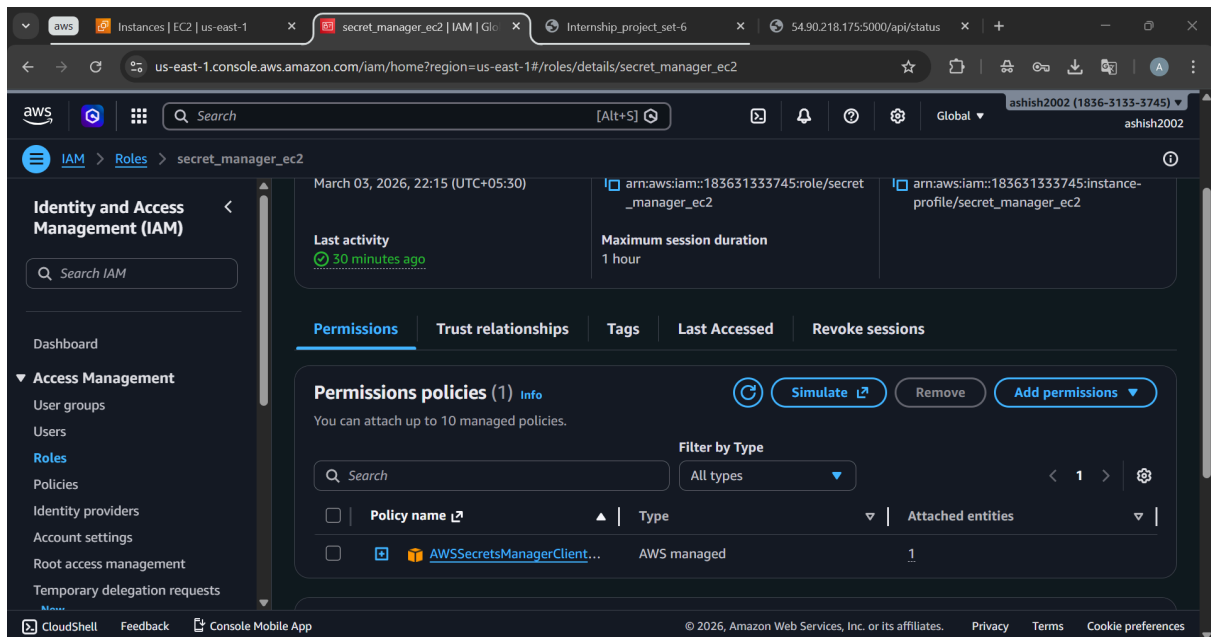
```
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "ashish@2002",
  database: "testdb"
});
```

Hardcoded credentials.

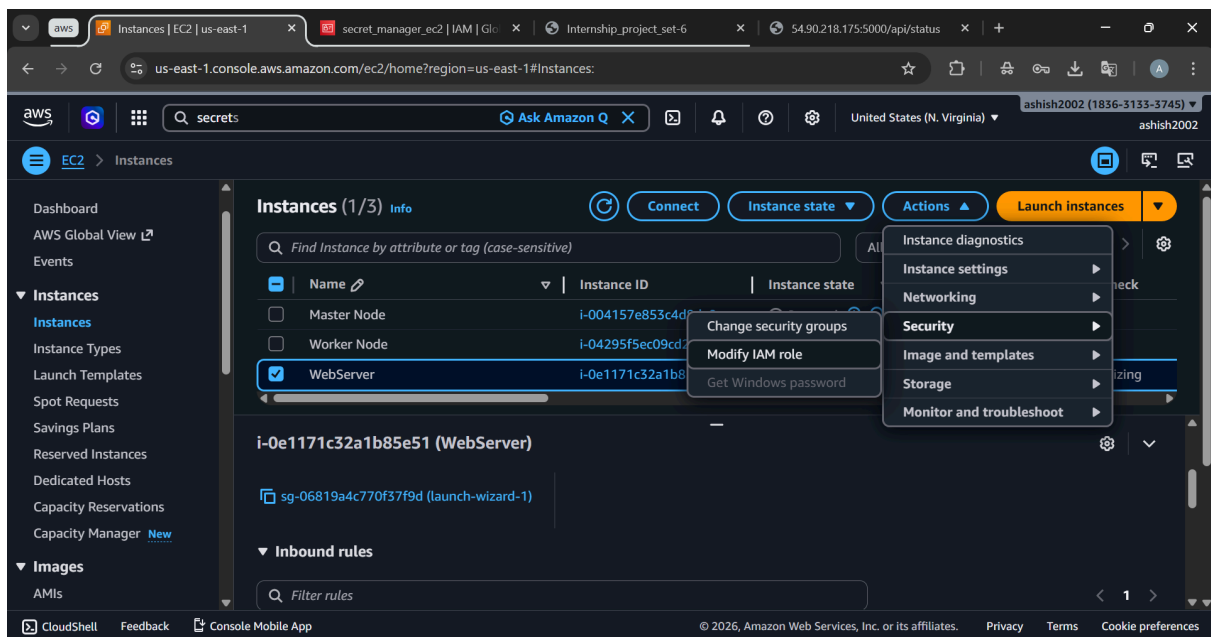
Now Instead of Hardcode we have to use AWS Secret Manager.
Create Secret Manager.

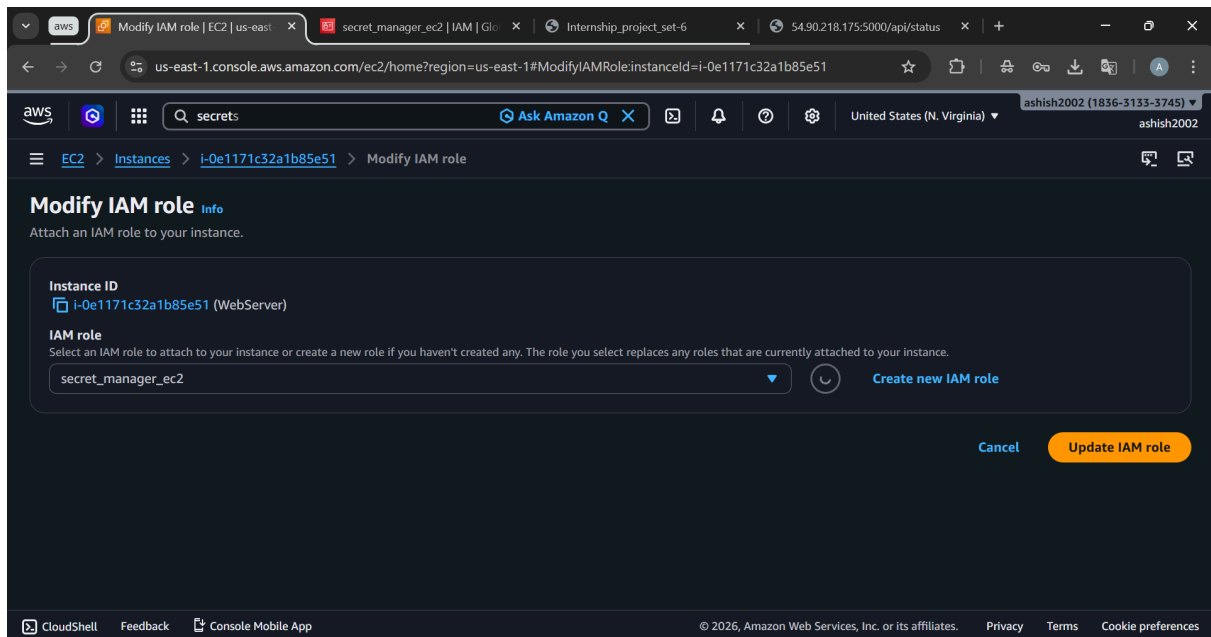


Create IAM Role.



And attach the role to the EC2.





Now Go to backend folder and install AWS SDK.

```
cd vite-node-mysql-app/backend
npm install @aws-sdk/client-secrets-manager@3.450.0
```

Update the server.js

```
const express = require("express");
const cors = require("cors");
const mysql = require("mysql2");
const { SecretsManagerClient, GetSecretValueCommand } = require("@aws-sdk/client-secrets-manager");

const app = express();
app.use(cors());

const client = new SecretsManagerClient({ region: "us-east-1" });

async function getDBConnection() {

    const command = new GetSecretValueCommand({
        SecretId: "DBSecret"
    });
}
```

```

    const response = await client.send(command);
    const secret = JSON.parse(response.SecretString);

    console.log("Secret response:", response);

    console.log("Parsed secret:", secret);
    const db = mysql.createConnection({
      host: "localhost",
      user: secret.database_username,
      password: secret.database_password,
      database: "testdb"
    });

    return db;
  }

  app.get("/api/status", async (req, res) => {

    try {

      const db = await getDBConnection();

      db.connect(err => {
        if (err) {
          res.json({
            status: "FAILED",
            message: "MySQL connection failed"
          });
        } else {
          res.json({
            status: "SUCCESS",
            message: "Connected using AWS Secrets Manager"
          });
        }
      });
    }
  });

```

```

    } catch (error) {

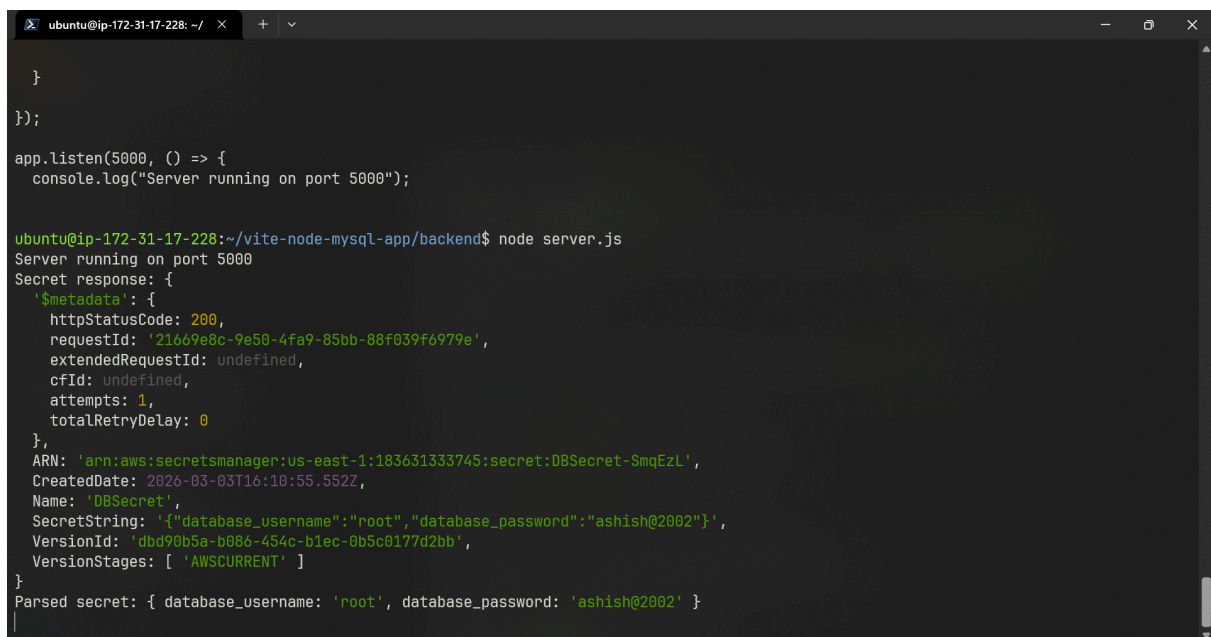
      res.json({
        status: "FAILED",
        message: "Error fetching secret from AWS"
      });

    }

  });

  app.listen(5000, () => {
    console.log("Server running on port 5000");
  });

```



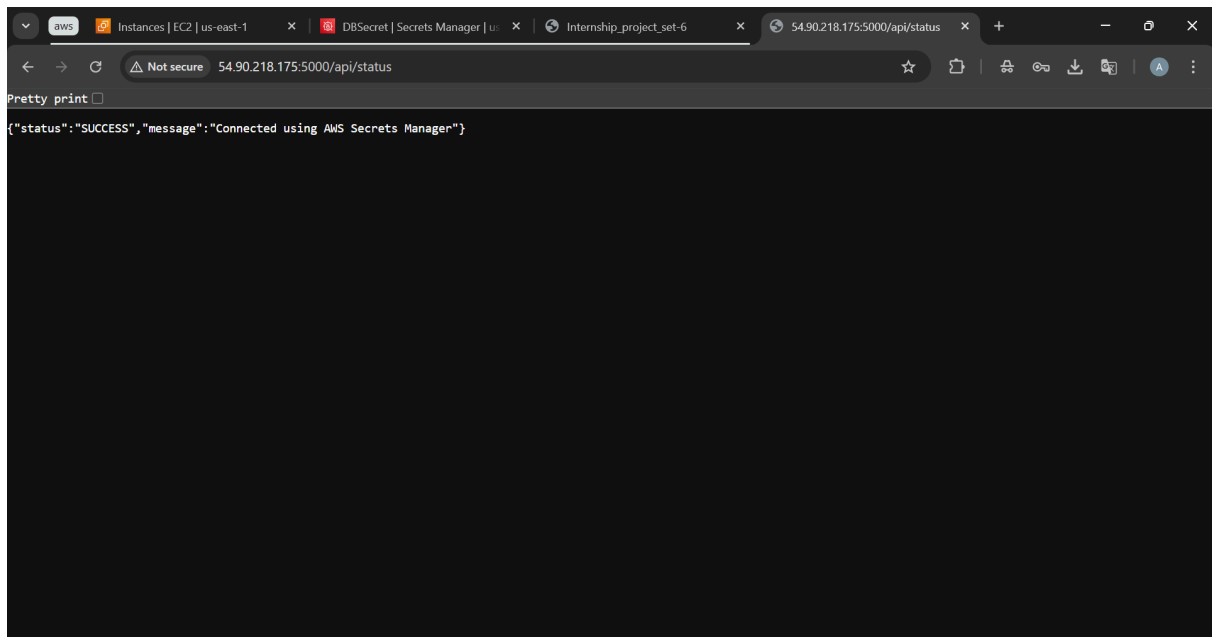
```

}
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});

ubuntu@ip-172-31-17-228: ~/vite-node-mysql-app/backend$ node server.js
Server running on port 5000
Secret response: {
  '$metadata': {
    httpStatusCode: 200,
    requestId: '21669e8c-9e50-4fa9-85bb-88f039f6979e',
    extendedRequestId: undefined,
    cfId: undefined,
    attempts: 1,
    totalRetryDelay: 0
  },
  ARN: 'arn:aws:secretsmanager:us-east-1:183631333745:secret:DBSecret-SmqEzL',
  CreatedDate: 2026-03-03T16:10:55.552Z,
  Name: 'DBSecret',
  SecretString: '{"database_username":"root","database_password":"ashish@2002"}',
  VersionId: 'dbd90b5a-b086-454c-b1ec-0b5c0177d2bb',
  VersionStages: [ 'AWSCURRENT' ]
}
Parsed secret: { database_username: 'root', database_password: 'ashish@2002' }

```



readme

